

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Applied Business Analytics Practical

Practical – X

Roll No.: T005	Name: Ritesh Chaubey
Class: TYIT	Batch: 1
Date of Assignment: 03-10-2026	Date/Time of Submission: 03-10-2026

AIM : Regression Analysis and Time Series Analysis

A) Develop a regression model to predict future sales based on historical data, assess model performance, and interpret the significance of predictor variables.

1. Load the sales_data.csv dataset.

```
import pandas as pd

df = pd.read_csv("sales_data.csv")

print("Dataset loaded successfully!")
```

✓ 0.1s

Dataset loaded successfully!

2. Display the first five records.

CODE :-

```
print(df.head())
```

OUTPUT :-

```
✓ 0.0s
```

	Date	Day	Month	Year	Customer_Age	Age_Group \
0	2013-11-26	26	November	2013	19	Youth (<25)
1	2015-11-26	26	November	2015	19	Youth (<25)
2	2014-03-23	23	March	2014	49	Adults (35-64)
3	2016-03-23	23	March	2016	49	Adults (35-64)
4	2014-05-15	15	May	2014	47	Adults (35-64)

3. Identify the independent and dependent variables.

Independent Variable: Advertising

Dependent Variable: Sales

4. Select advertising as the independent variable and sales as the dependent variable.

CODE :-

```
X = df[["Cost"]]
```

```
y = df[["Revenue"]]
```

```
print("Independent Variable:")
```

```
print(X.head())
print("\nDependent Variable:")
print(y.head())
```

OUTPUT :-

```
... Independent Variable:
      Cost
0    360
1    360
2   1035
3    900
4    180

Dependent Variable:
0    950
1    950
2   2401
3   2088
4    418
Name: Revenue, dtype: int64
```

5. Check the data for missing values.

CODE :

```
print(df.isnull().sum())
```

OUTPUT :-

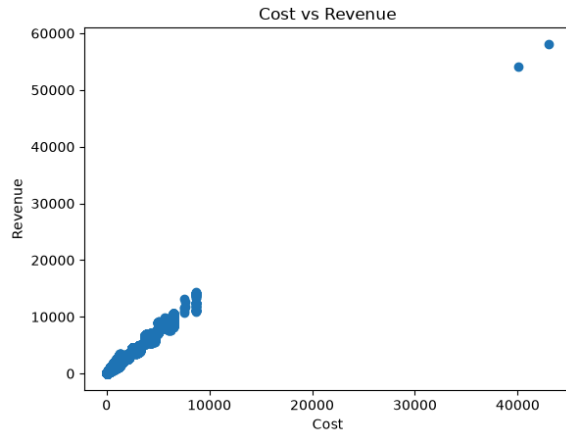
```
Date          0
Day           0
Month         0
Year          0
Customer_Age  0
Age_Group     0
Customer_Gender 0
Country       0
```

6. Create a scatter plot of advertising expenditure and sales.

CODE :-

```
import matplotlib.pyplot as plt
plt.scatter(df["Cost"], df["Revenue"])
plt.title("Cost vs Revenue")
plt.xlabel("Cost")
plt.ylabel("Revenue")
plt.show()
```

OUTPUT :-



7. Develop a simple linear regression model.

CODE :-

```
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(X, y)
print("Linear Regression Model Created Successfully")
```

OUTPUT :-

```
Linear Regression Model Created Successfully
```

8. Display the regression equation.

CODE :-

```
intercept = model.intercept_
coefficient = model.coef_[0]
print("Regression Equation:")
print(f'Revenue = {intercept:.2f} + ({coefficient:.2f} × Cost)')
```

OUTPUT :-

```
Regression Equation:
Revenue = 67.85 + (1.46 × Cost)
```

9. Use the model to predict sales for a given advertising expenditure.

CODE :-

```
cost_value = 1000
predicted_revenue = model.predict([[cost_value]])
print("Cost:", cost_value)
print("Predicted Revenue:", round(predicted_revenue[0], 2))
```

OUTPUT :-

```
Cost: 1000
Predicted Revenue: 1530.65
```

10. Calculate the R2 score of the model.

CODE :-

```
from sklearn.metrics import r2_score
y_pred = model.predict(X)
r2 = r2_score(y, y_pred)
print("R2 Score:", round(r2, 4))
```

OUTPUT :-

```
R2 Score: 0.9776
```

11. Calculate the Mean Squared Error (MSE).**CODE :-**

```
from sklearn.metrics import mean_squared_error
mse = mean_squared_error(y, y_pred)
print("Mean Squared Error (MSE):", round(mse, 2))
```

OUTPUT :-

```
Mean Squared Error (MSE): 38313.2
```

12. Compare the actual sales with the predicted sales.**CODE :-**

```
comparison = pd.DataFrame({
    "Actual Revenue": y,
    "Predicted Revenue": y_pred
})
print(comparison.head(10))
```

OUTPUT :-

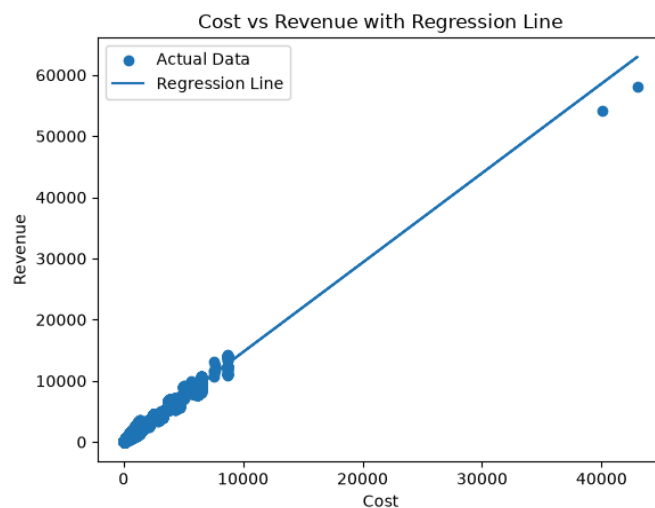
	Actual Revenue	Predicted Revenue
0	950	594.459443
1	950	594.459443
2	2401	1581.846542
3	2088	1384.369122
4	418	331.156216
5	522	396.982023
6	379	331.156216
7	190	199.504603
8	2086	1516.020736
9	1991	1450.194929

13. Plot the regression line with the actual data points.**CODE :-**

```
plt.scatter(
    df["Cost"],
    df["Revenue"],
    label="Actual Data"
)
```

```
plt.plot(  
    df["Cost"],  
    y_pred,  
    label="Regression Line"  
)  
plt.title("Cost vs Revenue with Regression Line")  
plt.xlabel("Cost")  
plt.ylabel("Revenue")  
plt.legend()  
plt.show()
```

OUTPUT :-



14. Interpret the regression results.

The regression analysis shows the relationship between Cost and Revenue. The regression coefficient indicates the expected change in Revenue for a one-unit increase in Cost. The R^2 score indicates how well Cost explains the variation in Revenue. The MSE represents the average squared difference between actual and predicted Revenue.

15. Write a conclusion about how advertising expenditure is related to sales.

Conclusion: The analysis shows a relationship between Cost and Revenue in the sales dataset. As Cost changes, Revenue also tends to change. The linear regression model can therefore be used to estimate Revenue based on Cost. The R^2 score and MSE help evaluate the performance and accuracy of the regression model.

B) Apply time series analysis techniques (e.g., ARIMA, exponential smoothing) to forecast future demand for a product or service, and evaluate the accuracy of the forecasts.

Part B: Time Series Analysis

Dataset: monthly_sales.csv

The dataset can contain:

Month	Sales
Jan	120
Feb	135
Mar	128
Apr	145
May	150
Jun	160
Jul	155
Aug	170
Sept	165
Oct	180
Nov	190
Dec	200

1. Load monthly sales data.**CODE :-**

```
import pandas as pd
df = pd.read_csv("monthly_sales.csv")
print(df.head())
```

OUTPUT :-

	date	sales
0	2013-01	14236.90
1	2013-02	4519.89
2	2013-03	55691.01
3	2013-04	28295.35
4	2013-05	23648.29

2. Convert the date column into date format.**CODE :-**

```
df["date"] = pd.to_datetime(df["date"])
print(df.dtypes)
```

OUTPUT :-

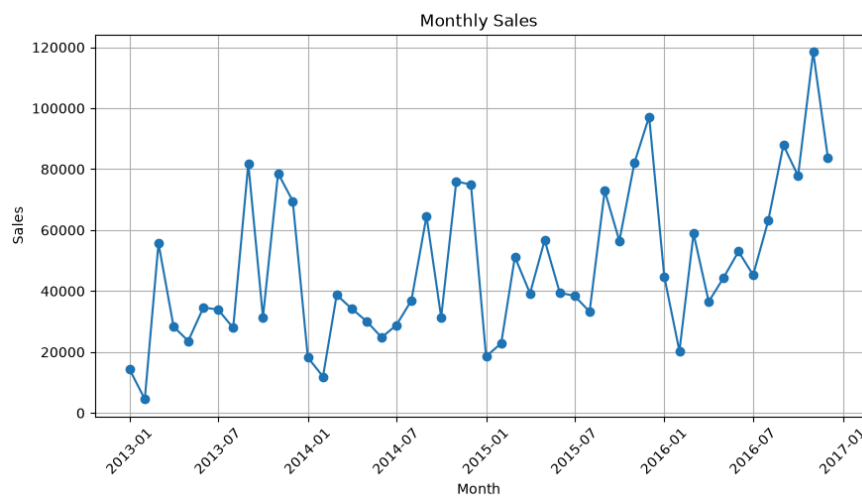
```
date      datetime64[us]
sales      float64
dtype: object
```

3. Plot monthly sales using a line chart.

CODE :

```
import matplotlib.pyplot as plt
plt.figure(figsize=(10, 5))
plt.plot(df["date"], df["sales"], marker="o")
plt.title("Monthly Sales")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.xticks(rotation=45)
plt.grid(True)
plt.show()
```

OUTPUT :-



4. Observe the trend.

The sales data shows an overall increasing trend from 2013 to 2016. Sales fluctuate considerably from month to month, with relatively high sales often occurring toward the end of the year.

5. Apply Exponential Smoothing.

CODE :-

```
from statsmodels.tsa.holtwinters import ExponentialSmoothing
# Create time series
sales_data = df.set_index("date")["sales"]
# Apply Exponential Smoothing
es_model = ExponentialSmoothing(
    sales_data,
    trend="add",
```

```
seasonal="add",
seasonal_periods=12
).fit()
print("Exponential Smoothing model applied successfully.")
```

OUTPUT :-

```
Exponential Smoothing model applied successfully.
c:\Users\mvluc\AppData\Local\Programs\Python\Python313\Li
self, init_dates(dates, freq)
```

6. Forecast the next 3 months.

CODE :-

```
es_forecast = es_model.forecast(3)
print("Exponential Smoothing Forecast:")
print(es_forecast)
```

OUTPUT :-

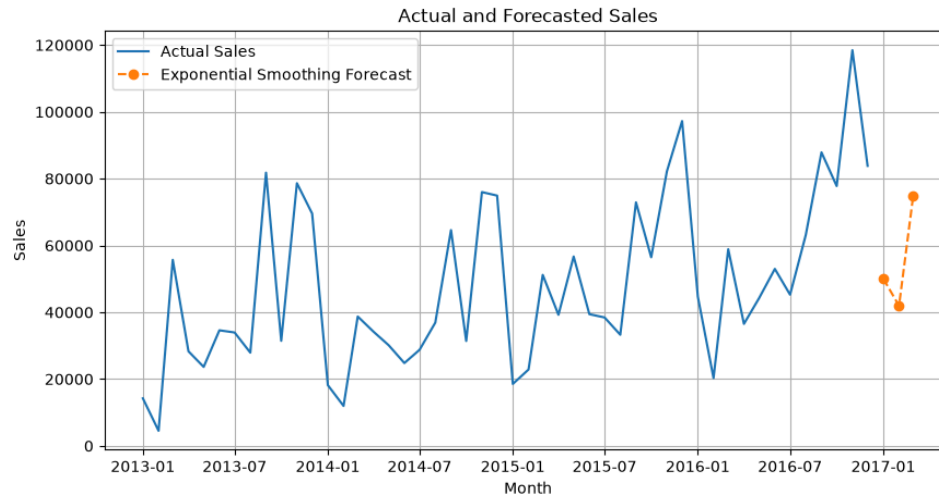
```
Exponential Smoothing Forecast:
2017-01-01    50053.289207
2017-02-01    41934.047835
2017-03-01    74892.673128
Freq: MS, dtype: float64
```

7. Plot actual and forecasted sales.

CODE :-

```
plt.figure(figsize=(10, 5))
plt.plot(
    sales_data,
    label="Actual Sales"
)
plt.plot(
    es_forecast,
    marker="o",
    linestyle="--",
    label="Exponential Smoothing Forecast"
)
plt.title("Actual and Forecasted Sales")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.legend()
plt.grid(True)
plt.show()
```

OUTPUT :-



8. Calculate forecasting error.

CODE :-

```
from statsmodels.tsa.holtwinters import ExponentialSmoothing
# Training data: first 45 months
train = sales_data.iloc[:-3]
# Test data: last 3 months
test = sales_data.iloc[-3:]
# Fit model on training data
es_test_model = ExponentialSmoothing(
    train,
    trend="add",
    seasonal="add",
    seasonal_periods=12
).fit()
# Forecast 3 months
es_test_forecast = es_test_model.forecast(3)
# Calculate MAE
es_error = abs(test - es_test_forecast).mean()
print("Exponential Smoothing MAE:",
      round(es_error, 2))
```

OUTPUT :-

```
Exponential Smoothing MAE: 19319.4
```

9. Apply ARIMA to the same data.

CODE :-

```
from statsmodels.tsa.arima.model import ARIMA
# Create ARIMA model
arima_model = ARIMA(
```

```

sales_data,
order=(1, 1, 1)
).fit()
print("ARIMA model applied successfully.")

```

OUTPUT :-

```

ARIMA model applied successfully.
c:\Users\mvluc\AppData\Local\Programs\
self._init_dates(dates, freq)
c:\Users\mvluc\AppData\Local\Programs\
self._init_dates(dates, freq)
c:\Users\mvluc\AppData\Local\Programs\
self._init_dates(dates, freq)

```

10. Forecast the next 3 months using ARIMA.

CODE :-

```

arima_forecast = arima_model.forecast(steps=3)
print("ARIMA Forecast:")
print(arima_forecast)

```

OUTPUT :-

```

ARIMA Forecast:
2017-01-01    75212.431581
2017-02-01    73212.732963
2017-03-01    72748.668167
Freq: MS, Name: predicted_mean, dtype: float64

```

11. Compare the ARIMA and Exponential Smoothing forecasts.

CODE :-

```

comparison = pd.DataFrame({
    "Exponential_Smoothing": es_forecast.values,
    "ARIMA": arima_forecast.values
}, index=["Jan 2017", "Feb 2017", "Mar 2017"])
print(comparison)

```

OUTPUT :-

	Exponential_Smoothing	ARIMA
Jan 2017	50053.289207	75212.431581
Feb 2017	41934.047835	73212.732963
Mar 2017	74892.673128	72748.668167

12. Calculate and compare their forecasting errors.

CODE :-

```

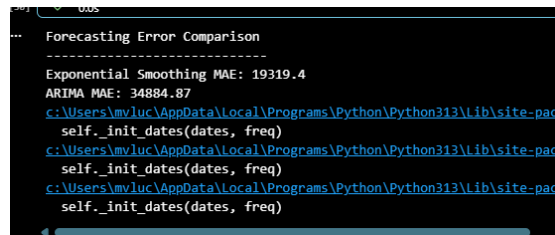
# ARIMA test model
arima_test_model = ARIMA(
    train,

```

```

    order=(1, 1, 1)
).fit()
# ARIMA prediction for test period
arima_test_forecast = arima_test_model.forecast(steps=3)
# Calculate errors
es_mae = abs(test - es_test_forecast).mean()
arima_mae = abs(test - arima_test_forecast).mean()
print("Forecasting Error Comparison")
print("-----")
print("Exponential Smoothing MAE:",
      round(es_mae, 2))
print("ARIMA MAE:",
      round(arima_mae, 2))

```

OUTPUT :-


```

Forecasting Error Comparison
-----
Exponential Smoothing MAE: 19319.4
ARIMA MAE: 34884.87
c:\Users\mv\luc\AppData\Local\Programs\Python\Python313\lib\site-pac
self._init_dates(dates, freq)
c:\Users\mv\luc\AppData\Local\Programs\Python\Python313\lib\site-pac
self._init_dates(dates, freq)
c:\Users\mv\luc\AppData\Local\Programs\Python\Python313\lib\site-pac
self._init_dates(dates, freq)

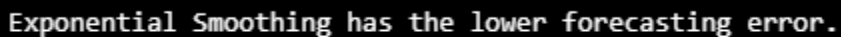
```

13. Identify which model gives the lower forecasting error for the given dataset.**CODE :-**

```

if es_mae < arima_mae:
    print("Exponential Smoothing has the lower forecasting error.")
else:
    print("ARIMA has the lower forecasting error.")

```

OUTPUT :-


```

Exponential Smoothing has the lower forecasting error.

```

14. Write the final conclusion.

The monthly sales data shows an overall increasing trend with noticeable monthly fluctuations and seasonal patterns. Both Exponential Smoothing and ARIMA were applied to forecast future sales. Based on the Mean Absolute Error (MAE) calculated using the last three months as test data, Exponential Smoothing produced a lower forecasting error than ARIMA for this dataset. Therefore, Exponential Smoothing provided the more accurate forecast among the two models for the given data.